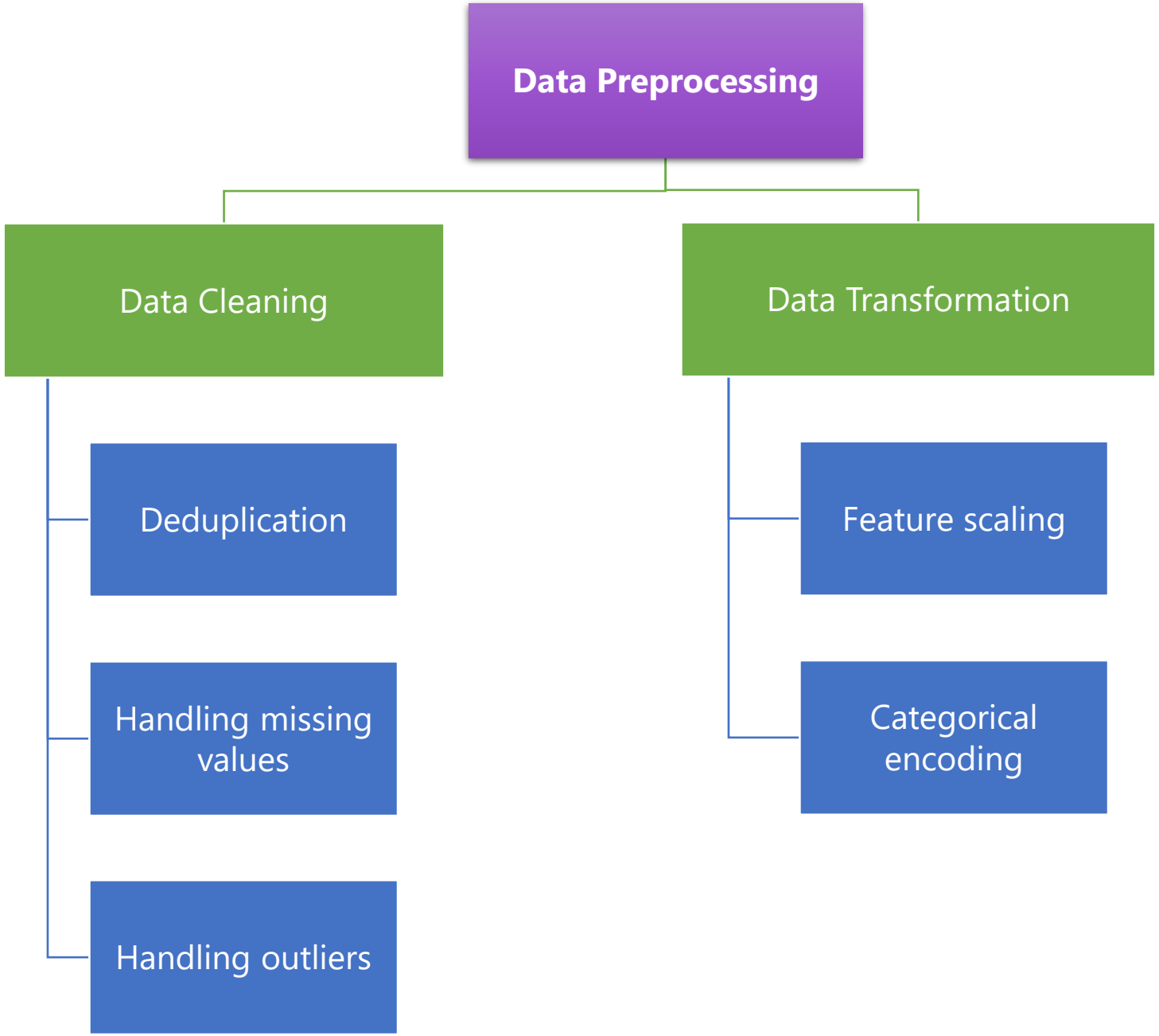
Data Preparing for Machine Learning



Outline


Data Preprocessing
Data Cleaning
Data Transformation
Deduplication
Handling missing values
Handling outliers
Feature scaling
Categorical encoding

Data Deduplication





05.dp**01-deduplication**

Data Deduplication

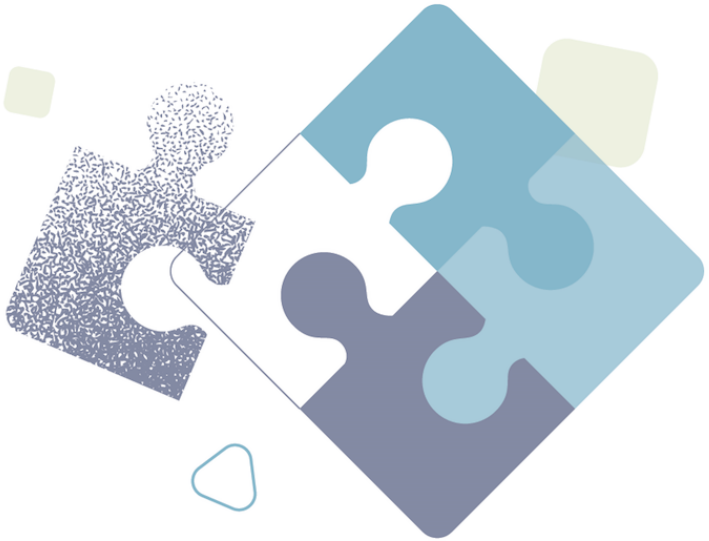
- **Duplicates:** instances with identical feature values, representing the same real-world entity or event

- Duplicate data can **bias ML models**, as repeated instances receive more influence and may distort learning outcomes. E.g.,
  - The same person submits an online form multiple times
  - Retweets share the same content as the original tweet, differing only in metadata (e.g., user, timestamp)

- Duplicate detection techniques:
  - **Exact matching**: direct comparison of records
  - **Similarity-based methods**: clustering or distance metrics to identify near-duplicates

```
# Deduplicate based on all columns except 'order_id'
df_deduped = df.drop_duplicates(subset=df.columns.difference(['order_id']))
```

Handling Missing Values





05.dp\02-handling-missing-values

Missing Values: The Big Picture

1 Why is it missing?

MCAR

Completely at random

Pure chance, e.g. a sensor fails or a survey page is not completed. Safest case to impute.

MAR

At random (given other columns)

Depends on other features, e.g. income is missing more often for younger people.

MNAR

Not at random

Depends on the value itself, e.g. very high earners skip the income question.

2 How much is missing in a column?

< 5%	Simple imputation (mean / median / mode) is usually safe
5 – 30%	Impute and add a missing indicator; try model-based imputation
30 – 60%	Be careful: heavy imputation distorts the data. Ask a domain expert
> 60%	Consider dropping the column

Step 1: Find the Missing Values

- **Count and percentage** per column with **isna()** shows where the gaps are
- **Hunt for disguised missing values:** "NA", "n/a", "null", "_" or codes like -99 look like data, so pandas will not count them
- **Ask why:** random, related to other columns, or related to the value itself? The answer picks the fix

Python | pandas

```python
import pandas as pd
import numpy as np

# 1. Count and % of missing values per column
report = pd.DataFrame({
    "missing": df.isna().sum(),
    "pct": (df.isna().mean() * 100).round(1),
})
print(report.sort_values("pct", ascending=False))

# 2. Turn "disguised" missing values into real NaN
fake_na = ["NA", "n/a", "None", "null", "_", "",
"empty"]
df = df.replace(fake_na, np.nan)
df["parking_space"] = df["parking_space"].replace(-99,
np.nan)
```

Step 2: Drop What You Can't Fix

- **Drop columns** that are mostly empty (> 60% missing): they add noise and little signal
- **Drop rows** only when a critical value is missing, e.g. the target you want to predict
- **Be careful:** if many rows are affected, dropping throws away too much data. Impute instead

Python | pandas

```python
# Drop columns with more than 60% missing values
too_empty = df.columns[df.isna().mean() > 0.60]
df = df.drop(columns=too_empty)
print("Dropped:", list(too_empty))

# Drop rows that miss a critical value (e.g. the target)
df = df.dropna(subset=["income"])

# Check what is left
print(df.shape)
print(df.isna().sum())
```

Rule of thumb: remove a column only if it is missing > 60% of its values; for less missing data use imputation (next steps).

Step 3: Mean / Median / Mode Imputation

- **Numeric:** mean if roughly symmetric, **median** if skewed or has outliers
- **Categorical: mode** (most frequent value)
- **Use when** data is missing at random and at most ~5% per column
- **Limitation:** it shrinks variance and distorts correlations; more missing means more distortion

Avoid data leakage: fit the imputer on the training set only, then transform the test set.

Python | scikit-learn

```python
from sklearn.impute import SimpleImputer

num_imp = SimpleImputer(strategy="median")       # or "mean"
cat_imp = SimpleImputer(strategy="most_frequent")  # mode

# fit on TRAIN only, then apply to train and test
X_train[num_cols] = num_imp.fit_transform(X_train[num_cols])
X_test[num_cols]  = num_imp.transform(X_test[num_cols])

X_train[cat_cols] = cat_imp.fit_transform(X_train[cat_cols])
X_test[cat_cols]  = cat_imp.transform(X_test[cat_cols])

# Quick pandas version (for exploration only)
df["age"] = df["age"].fillna(df["age"].median())
```

Step 4: When “Missing” Means Something

- **Missing indicator:** a 0/1 column saying “was missing”; the gap itself may predict the target
- **Constant value:** a clearly fake number (-99) for “not applicable”, or a “high” guess when high earners hide income (MNAR)
- **“Missing” category:** one explicit label for all categorical gaps
- **Interpolate:** for ordered or time-series data, use the previous and next values

Python | pandas

```python
# 1. Missing indicator (flag) + dummy value
df["parking_missing"] =
df["parking_space"].isna().astype(int)
df["parking_space"] = df["parking_space"].fillna(-99)

# 2. MNAR: high earners skip income -> fill with a
"high" guess
df["income"] = df["income"].fillna(250)

# 3. Categorical: explicit "Missing" label
df["entree"] = df["entree"].fillna("Missing")

# 4. Ordered data: average of previous and next value
df["years_seniority"] =
df["years_seniority"].interpolate(method="linear")
```

Tip: tree-based models can use the flag + dummy value directly.

Putting It Together: Imputation Pipeline

Python | scikit-learn

```python
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

num_cols = ["age", "income", "parking_space"]
cat_cols = ["entree", "pets"]

prep = ColumnTransformer([
    ("num", SimpleImputer(strategy="median",
                add_indicator=True), num_cols),
    ("cat", SimpleImputer(strategy="constant",
                fill_value="Missing"), cat_cols),
])

X_train_clean = prep.fit_transform(X_train)# learn on train
X_test_clean  = prep.transform(X_test)    # reuse on test
```

Why a pipeline?

No leakage
the imputer learns only from training data

Repeatable
same steps for train, test and new data

Flags for free
add_indicator=True adds the missing flags

Need smarter?
try KNNImputer or IterativeImputer, but start simple

Summary: Choosing a Strategy

Random (MCAR / MAR)
Missingness is not about the value itself

Numeric	Categorical
Mean or median	**Mode**

Not random (MNAR)
Missingness is about the value itself

Numeric	Categorical
Arbitrary / extreme value	**“Missing” category**

In both cases, add a missingness indicator when the missingness itself may carry information.

1

Inspect first
isna() and disguised values

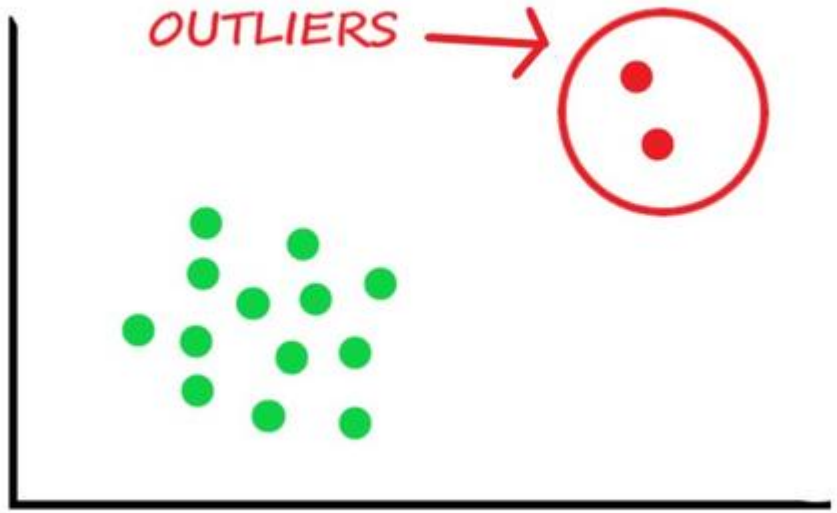
2

Start simple
drop > 60%, impute the rest

3

Fit on train only
avoid data leakage

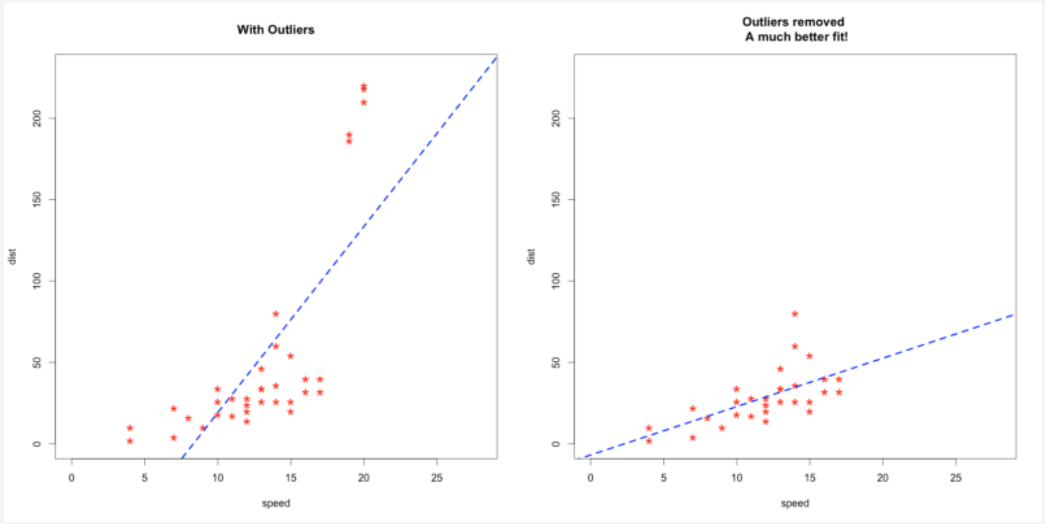
Handling Outliers





05.dp\3-handling-outliers

Step 1: Spot the Outliers



https://www.r-bloggers.com/outlier-detection-and-treatment-with-r/

- **Outlier:** a value far from the rest. It may be a real rare case or an error
- **Sensitive models:** linear / logistic regression and distance-based models. Trees are robust
- **Never delete blindly:** check with domain knowledge first

Python | pandas

```python
import pandas as pd

# 1. Visual check: boxplot and histogram
df["income"].plot.box()
df["income"].plot.hist(bins=30)

# 2. IQR rule (works well for skewed data)
q1, q3 = df["income"].quantile([0.25, 0.75])
iqr = q3 - q1
low  = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr

# 3. Count and inspect the outliers
mask = (df["income"] < low) | (df["income"] > high)
print(mask.sum(), "outliers")
print(df[mask])
```

Tip: the boxplot whiskers use this same IQR rule, so plot first, then count.

Step 2: Choose How to Handle Them

A **Trim (remove)**
Pro: fast and simple. Con: loses rows and information. Use for few, clearly wrong values.

B **Cap (winsorize)**
Pro: keeps every row. Con: slightly distorts the distribution. Most common choice (next slide).

C **Treat as missing**
Pro: no data removed; reuse imputation. Con: may distort the distribution.

D **Bin (discretize)**
Pro: extremes fall into the first or last bin. Con: loses fine detail.

Python | pandas

```python
import numpy as np

# A. Trim: keep only the rows that are NOT outliers
df_trim = df[~mask]

# B. Cap: clip values to the IQR limits (next slide)
df["income_cap"] = df["income"].clip(low, high)

# C. Treat as missing, then impute with the median
df["income_nan"] = df["income"].mask(mask)
med = df["income_nan"].median()
df["income_nan"] = df["income_nan"].fillna(med)

# D. Bin: 5 equal-sized groups, extremes land in the
#    first or last bin
df["income_bin"] = pd.qcut(df["income"], q=5,
                           labels=False, duplicates="drop")
```

Step 3: Capping (Winsorization)

Replace extreme values with a limit instead of deleting them:

IQR

Skewed data

e.g. income, house prices

Lower = Q1 − 1.5 × IQR

Upper = Q3 + 1.5 × IQR

Z-score

Roughly normal data

e.g. height

Lower = mean − 3σ

Upper = mean + 3σ

Result: below lower limit → lower limit; above upper → upper limit.

Avoid leakage: compute limits on the training set only, then reuse them on the test set.

Python | pandas

```python
# Option 1: IQR rule (skewed data, e.g. income)
q1, q3 = df["income"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df["income"] = df["income"].clip(lower=low,
        upper=high)

# Option 2: z-score rule (roughly normal, e.g. height)
m, s = df["height"].mean(), df["height"].std()
df["height"] = df["height"].clip(lower=m - 3 * s,
                                 upper=m + 3 * s)

# Check the effect
print(df[["income", "height"]].describe())
```

Tip: cap only if outliers hurt your model. Compare scores with and without capping.

Feature Scaling

Standardisation	Normalisation
$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$	$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$



05.dp\04-feature-scaling

Feature Magnitude matters

- Continuous features that cover very different ranges (i.e., Magnitude) can cause difficulty for some ML algorithms that are sensitive to feature magnitude
  - e.g., Ages range [16, 96], whereas Salaries range [10,000, 100,000]—features with large values (scales) dominate

Person	Age	Salary
1	20	25,000
2	65	25,500
3	25	25,700
4	22	26,900
5	55	25,400

- Person 1 is closer to Person 3 than Person 2, however, some distance functions will say that Person 1 and Person 2 are more similar
- Example: $\boldsymbol{distance(x, y) = \sum_{i=1}^{n} |x_i - y_i|}$

  |x_Age - y_Age| + |x_Salary - y_Salary|

  - distance(1, 2) = 45 + 500 = 545
  - distance(1, 3) = 5 + 700 = **705**

Feature scaling

- Feature scaling is the process of transforming feature values measured on different scales to **a common scale**
  - It is typically the final step in the data preprocessing pipeline, applied just before training the ML model
  - Feature scaling does not change the shape of the data distribution
- Why Feature Scaling is important:
  - Prevents features with large numeric ranges from dominating the model
  - Makes features comparable
  - Improves performance of distance-based algorithms (e.g., KNN, k-means, PCA)
  - Speeds up convergence of gradient-based optimization (e.g., neural networks)

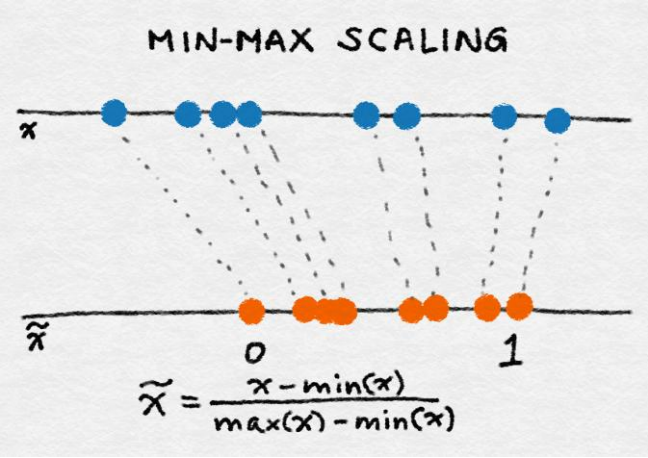
Min-Max Scaling

- **Scales the variable between 0 and 1**

$$x_i' = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

x_i' is the scaled value

- Aka **Normalization**
- Sensitive to outliers



Price
100
90
50
40
20
100
50
60
120
40
200

Max = 200
Min = 20
Range = 200 – 20 = 180

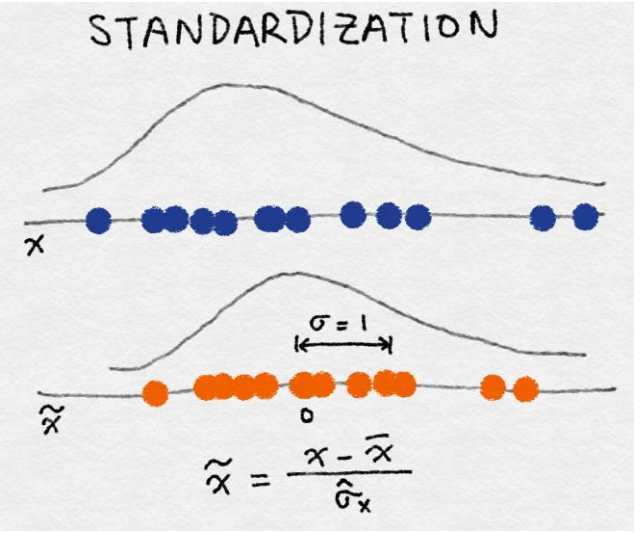
Obs. - Min

Range

Price
0.44
0.39
0.17
0.11
0.00
0.44
0.17
0.22
0.56
0.11
1.00

Image source

Standardization



- Standardization centers a feature at **zero mean** and scales it to **unit variance**

$$x_i' = \frac{x_i - \mu}{\sigma}$$

x_i' is the standardized feature value, x_i = original value, μ = mean for feature x, and σ = standard deviation

  - The standardized value indicates how many standard deviations a data point is from the mean
  - Transforms data to have mean = 0 and standard deviation = 1
  - Preserves outliers
  - Works best when the data is approximately normally distributed

Price
100
90
50
40
20
100
50
60
120
40
200

Mean = 79
Standard dev = 51

Obs. - Mean

Standard dev

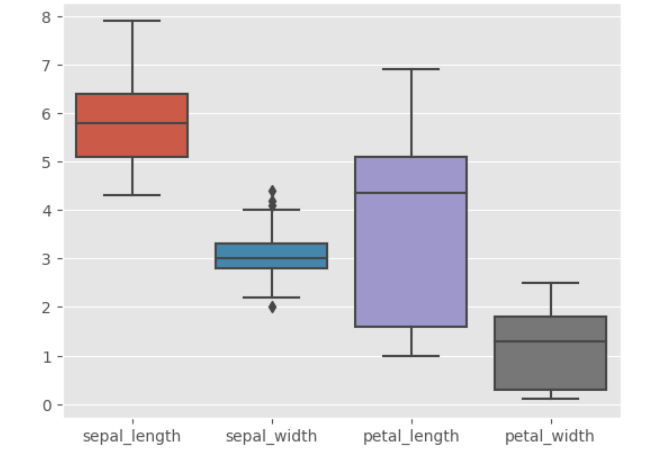
Price
0.41
0.22
-0.57
-0.76
-1.16
0.41
-0.57
-0.37
0.80
-0.76
2.37

Image source

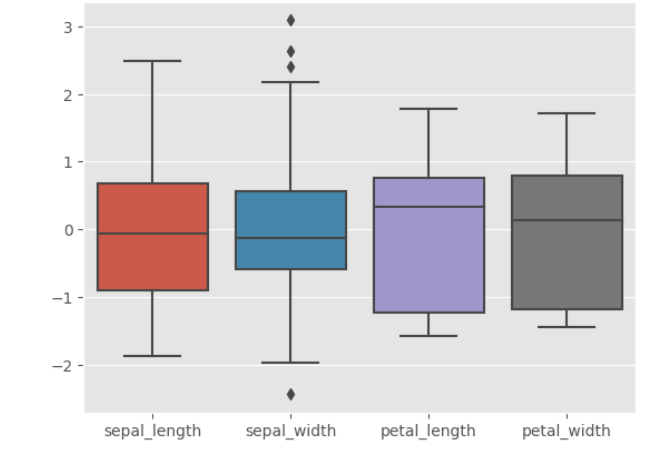
Feature scaling does not change the shape of the distribution

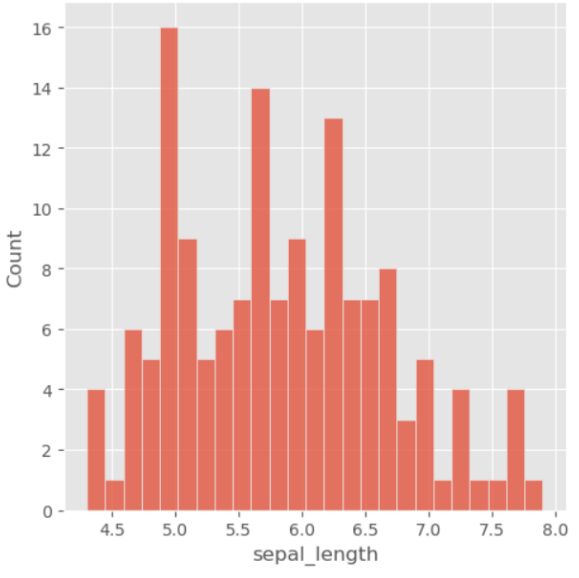
```
iris_scaled = iris.copy()
X = iris_scaled.values[:,0:-1]  #columns, except the class label, of the matrix of the dataframe
X = X.astype(float) #convert X's dtype from object to float
X_scaled = preprocessing.StandardScaler().fit_transform(X)
iris_scaled.iloc[:,0:-1] = X_scaled
iris_scaled.head()
```

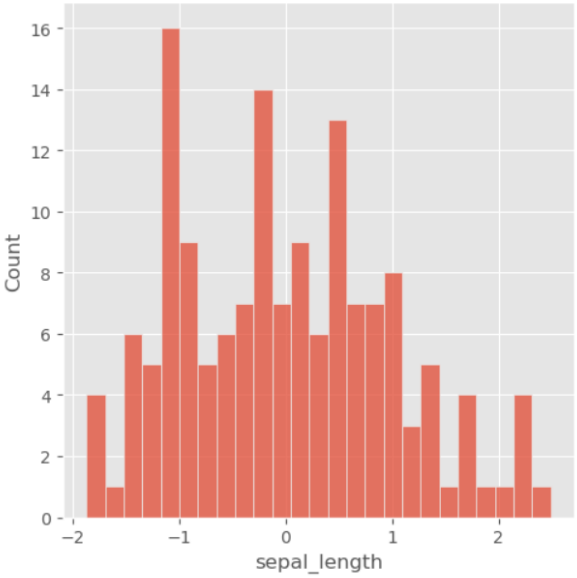
```
sns.boxplot(data=iris)
plt.show()
```




```
sns.boxplot(data=iris_scaled)
plt.show()
```

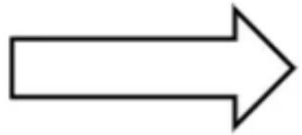






Categorical Variable Encoding

Grades
A
B
C
D
Fail



Grades	Encoded
A	4
B	3
C	2
D	1
Fail	0



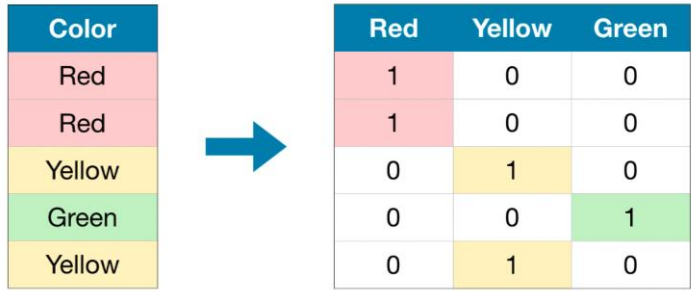
05.dp**05-categorical-encoding**

Categorical encoding

- Categorical encoding = Transform categorical (string) features into numeric representations so they can be used by ML models
- Common Encoding Methods:
  - **One hot** encoding: Creates a binary column for each category (used for Linear models)
  - **Ordinal / Label** encoding: Assigns an integer to each category (used for Tree based models)
  - **Count / frequency** encoding: Replaces each category with its frequency (used when there are many categories)

One hot encoding

- Converts a categorical feature into multiple **binary (0/1) indicator variables**, one per category
  - A value of **1** indicates the presence of the category, and **0** indicates its absence



Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- **When to Use:** Suitable for **linear models** and algorithms that assume no ordinal relationship between categories
- **Limitations**
  - **Large feature expansion** when a variable has many categories (high cardinality)
  - Can introduce **redundant or identical dummy variables**, adding no new information

Ordinal Encoding

- Assigns **each category a unique integer** value (e.g., 0 to n-1 or 1 to n, where n is the number of distinct categories)
  - 👍 Does not expand the feature space
- **When to Use**:
  - Works well with tree-based models, which can handle arbitrary integer labels
- **Limitations:**
  - Not suitable for linear models or distance-based algorithms, since they may incorrectly interpret the numeric labels as having meaningful order or magnitude

Color
Green
Red
Blue



Color
1
2
3

Count / frequency encoding

- Replaces each category with its **count** or **frequency** in the dataset
  - Does not expand the feature space

Color
Green
Red
Black
Green

Count encoding →

Encoding
2
1
1
2

When to Use

- Useful for **high-cardinality** categorical variables
- Works well with **tree-based models**

Limitations

- **Not suitable for linear models**
- Categories with the **same frequency get the same value**
  - Can lead to **loss of information**